INTERNSHIP PROJECT REPORT

Real-Time AI Surveillance System for Face and Aircraft Recognition

Submitted by

KONADA BRAHMI
B.TECH CSE
GITAM UNIVERSITY

STRATEGIC ELECTRICAL RESEARCH AND DESIGN(SLRDC),

HINDUSTAN AERONAUTICS LIMITED (HAL), HYDERABAD

Under the Guidance of
Shri Kumar Pushpendra, CM(D)
Duration: 06-05-2026 to 03-06-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **SLRDC, HAL, Hyderabad** for providing me with the opportunity to work on this internship project.

I sincerely thank **Shri Kumar Pushpendra** and **Mr. Gaurav Suryavanshi singh** for their valuable guidance, support, and encouragement throughout the internship period

I also thank all the staff members and technical team for their support and encouragement during the successful completion of this project.

ABSTRACT

Survilience systems are used to detect and continuously monitor moving objects such as aircraft and aerial targets in real time.

Inspired by these systems, the project "**Real-Time AI Surveillance System for Face and Aircraft Recognition**" was developed to perform continuous face recognition and aircraft detection using Artificial Intelligence.

The system uses YOLOv8, DeepFace, OpenCV, and Flask to achieve balanced real-time performance, detection accuracy, and continuous surveillance monitoring through a web-based dashboard.

CERTIFICATE

This is to certify that the internship project report entitled "**REAL-TIME AI SURVEILLANCE SYSTEM FOR FACE AND AIRCRAFT RECOGNITION**" submitted by **KONADA BRAHMI** of B. TECH CSE, GITAM UNIVERSITY, is a bona fide record of work carried out at SLRDC, HAL, Hyderabad under my supervision.

Shri Kumar Pushpendra
CM(D)
Mentor
SLRDC (HAL), HYDERABAD

DECLARATION

I hereby declare that the project titled "**Real-Time AI Surveillance System for Face and Aircraft Recognition**" submitted as part of the internship work carried out at HAL, Hyderabad, is an original work developed by me under the guidance and support provided during the internship period.

I further declare that this project has not been submitted previously, either fully or partially, for the award of any degree, diploma, or certification in any institution.

KONADA BRAHMI
B.Tech CSE
GITAM University

ABOUT THE ORGANIZATION

Hindustan Aeronautics Limited (HAL) is India's leading aerospace and defence company, wholly owned by the Government of India under the Ministry of Defence. Established in 1940, HAL designs, develops, manufactures, and maintains aircraft, helicopters, aero-engines, avionics, and related systems.

The **Strategic Electronic R&D Centre (SLRDC)** in Hyderabad is HAL's specialized AS 9100D-certified R&D center focused on **avionics and strategic electronics**. Co-located with HAL's Avionics Division in Balanagar, Hyderabad, SLRDC conducts advanced research and development in electronic systems for aircraft and defence platforms, contributing to India's self-reliance in aerospace technology.

HAL is involved in major projects like the Tejas fighter jet, LCH helicopters, and aerospace components for India's space missions.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Introduction to Recognition System

Recognition is the process of identifying and distinguishing a specific object, person, or target from a set of known entities using Artificial Intelligence and Computer Vision techniques. Recognition systems analyze unique features from images or video streams and compare them with stored reference data to determine the identity of the detected object.

Recognition technology is widely used in security systems, access control, surveillance applications, biometric authentication, aerospace monitoring, and defence systems. In modern AI-based surveillance environments, recognition systems help automate monitoring tasks, improve operational efficiency, and reduce dependence on manual observation.

1.2 Need of the Project in Aerospace and Aircraft Manufacturing Industry

The aerospace and aircraft manufacturing industry requires continuous monitoring of personnel, aircraft assets, and operational activities to ensure safety, security, and efficient management. Manual monitoring of surveillance feeds can be time-consuming and may result in delayed responses or human errors.

AI-based recognition systems can assist by automatically identifying authorized personnel, recognizing aircraft objects, maintaining surveillance records, and providing real-time monitoring through centralized dashboards. Such systems can support security operations, restricted-area monitoring, aircraft maintenance facilities, production environments, and aerospace testing centres.

With further optimization, similar AI-based recognition systems can also be deployed on drones and embedded platforms for aerial monitoring, perimeter surveillance, aircraft inspection, and remote security applications.

1.3 Project Overview

The "**Real-Time AI Surveillance System for Face and Aircraft Recognition**" is a computer vision-based surveillance platform developed to perform real-time face recognition and aircraft detection using Artificial Intelligence technologies. The project was developed during a four-week internship at Strategic Electronics Research and Design Centre (SLRDC), HAL Hyderabad with the objective of understanding modern surveillance systems and implementing an intelligent monitoring framework.

The system combines multiple technologies such as OpenCV, DeepFace, YOLOv8, Flask, HTML, CSS, and JavaScript to create a modular and real-time surveillance

environment. The applica tion is capable of detecting human faces, identifying registered individuals, and detecting aircraft objects through live video streams.

The project was designed with two major operational modes:

- Face Recognition Mode for identifying human subjects.
- Aircraft Detection Mode for monitoring aerial objects in real time.

The surveillance system processes live camera frames continuously, performs AI-based detection and recognition, and displays the outputs through a web-based dashboard interface. The dashboard provides real-time monitoring, detection visualization, and surveillance controls for smooth operation.

Problem Statement:
Manual monitoring of live surveillance feeds is time-consuming and may lead to delayed response or human error. There is a need for an intelligent system that can automatically detect aircraft objects and recognize human faces in real time, while also providing a simple and effective dashboard for continuous monitoring and control.

1.4 Objectives of the System

- To design a modular surveillance architecture that can contribute toward future development of advanced search and track systems used in aerospace and monitoring applications.
- To develop a real-time AI surveillance system capable of accurately recognizing human faces and detecting aircraft objects from live video streams.

2. TECHNOLOGIES USED AND SYSTEM ARCHITECTURE

2.1 Software and Libraries Used

The project was developed using multiple software technologies and Artificial Intelligence libraries to achieve real-time surveillance, face recognition, and aircraft detection functionalities. Each technology was selected based on its performance, flexibility, and suitability for real-time Computer Vision applications.

Python

Python was used as the primary programming language for the project due to its simplicity, extensive AI support, and availability of Computer Vision libraries. It was used to implement the backend logic, AI processing pipeline, and surveillance workflow.

OpenCV

OpenCV (Open-Source Computer Vision Library) was used for image and video processing operations.

It handled:
- Live webcam video capture
- Frame processing
- Image resizing
- Drawing bounding boxes
- Face region extraction
- Real-time display operations

OpenCV played a major role in handling continuous video streams efficiently and was also used as the face detector backend during recognition operations.

DeepFace

DeepFace was used for facial recognition and identity matching. Different DeepFace models were used for different surveillance operations:
- GhostFaceNet was used for real-time live surveillance recognition due to its balanced speed and performance.
- ArcFace was used for uploaded image recognition because of its high facial recognition accuracy and embedding quality.
- Facenet was used for aircraft image comparison and recognition experiments.

DeepFace generated facial embeddings and compared them with stored database images to identify matching targets.

YOLOv8

YOLOv8 (You Only Look Once Version 8) was used for real-time object detection.

It was responsible for detecting Persons and aircraft objects

YOLOv8 provided:
• Fast object localization
• Bounding box generation
• Real-time detection capability
• Balanced detection accuracy and speed

Flask

Flask was used to develop the web-based surveillance dashboard.

It handled:
• Web server operations
• Live video streaming
• Dashboard communication
• Surveillance controls
• File upload handling

Flask enabled the system to provide a real-time monitoring interface accessible through a web browser.

HTML, CSS, and JavaScript

Frontend technologies were used to design the surveillance dashboard interface.
• HTML was used to structure the webpage.
• CSS was used for styling and creating the dark-themed surveillance UI.
• JavaScript was used for interactive controls such as Start/Stop surveillance operations and real-time dashboard updates.

These technologies helped create a user-friendly and responsive monitoring environment.

Threading Module

Python's threading module was used to improve real-time performance by running video capture, AI inference, and dashboard operations simultaneously. Multi-threading reduced lag and improved overall system responsiveness.

Overall, the combination of these software technologies and libraries enabled the development of a modular, real-time AI surveillance system capable of face recognition and aircraft detection.

2.2 System Architecture Workflow

The system architecture was designed to perform real-time surveillance, face recognition, and aircraft detection using Artificial Intelligence and Computer Vision technologies. The architecture follows a modular workflow where live video frames are continuously captured, processed, analyzed using AI models, and displayed through a web-based surveillance dashboard.

The complete workflow of the system is shown below:

Step 1: Live Video Input

The surveillance system continuously captures live video frames from the webcam using OpenCV. These frames act as the primary input for the detection and recognition pipeline.

Step 2: Frame Preprocessing

Before sending frames to the AI models, preprocessing operations are performed to improve processing speed and reduce computational load.

The preprocessing steps include:

- Frame resizing to lower resolution
- Colour conversion when required
- Noise reduction
- Frame normalization
- Region of Interest (ROI) extraction

Frame resizing was mainly used to reduce CPU usage and improve real-time inference performance.

Step 3: Object Detection using YOLOv8

After preprocessing, the frames are passed to the **YOLOv8 (You Only Look Once Version 8)** object detection model.

YOLOv8 is a deep learning-based real-time object detection model that works by:

- Dividing the image into regions
- Detecting objects within a single forward pass
- Generating bounding boxes around detected objects
- Assigning confidence scores and class labels

In this project, YOLOv8 was used to detect:

- Persons
- Aircraft objects

The model performs fast real-time object localization and detection while maintaining balanced accuracy and speed.

Step 4: Face Recognition using DeepFace

When a person is detected by YOLOv8, the detected face region is extracted and sent to the DeepFace recognition module.

DeepFace works by:

- Extracting facial features from the detected face
- Generating face embeddings (numerical facial representations)
- Comparing these embeddings with stored database images
- Identifying the closest matching person

The system then labels the detected face with the corresponding identity name if a match is found.

Step 5: Real-Time Output Generation

After detection and recognition, the system generates:

- Bounding boxes
- Detection labels
- Recognition names
- Confidence outputs

These outputs are drawn directly onto the video frames in real time.

Step 6: Dashboard Display using Flask

The processed video stream is transmitted to the Flask-based surveillance dashboard. The dashboard allows users to:

- Monitor live surveillance feeds
- View real-time detections
- Start or stop surveillance operations
- Observe recognition outputs

Step 7: Continuous Real-Time Monitoring

The entire pipeline runs continuously in real time using multi-threaded execution. This enables smooth monitoring and simultaneous processing of:

- Face recognition
- Aircraft detection
- Live dashboard streaming

The modular architecture improves:

- Scalability
- Processing efficiency
- Real-time responsiveness
- Future system expansion capability

The workflow also allows independent upgrades to AI models and frontend components without affecting the complete system structure.

2.5 Features of the Surveillance Website

The major features of the surveillance website are as follows:

Real-Time Video Streaming

The website provides continuous live video streaming from the surveillance camera. The live feed is displayed directly on the dashboard for real-time monitoring.

Real-Time Face and Aircraft Recognition

The system continuously monitors live video streams to detect and recognize human faces and aircraft objects in real time. Human faces are recognized using DeepFace, while aircraft objects are detected using the YOLOv8 object detection model. Detected objects are highlighted with bounding boxes and corresponding detection labels on the surveillance dashboard.

Live Detection Display

The dashboard displays:

- Bounding boxes
- Detection labels
- Recognition names
- Real-time surveillance outputs

This allows users to monitor detections continuously.

File Upload and Recognition Feature

The website also includes a file upload feature that allows users to upload images for recognition and detection. Uploaded images are processed by the AI models to identify faces or detect aircraft objects. The detection results are then displayed directly on the dashboard interface.

Saved Log Data

The system stores detection information such as:

- Detected person names
- Aircraft detections
- Detection timestamps
- Recognition outputs

The saved data helps maintain surveillance records for future reference and monitoring purposes.

Downloadable Log Reports

The surveillance logs can be downloaded from the dashboard interface. This feature allows users to save and review detection records and monitoring data whenever required.

Start and Stop Surveillance Controls

The website includes operational control buttons that allow users to:

- Start surveillance
- Stop surveillance
- Control live monitoring operations

These controls improve usability and system management.

Modular System Design

The website architecture is modular, allowing future integration of:

- Advanced AI models
- Embedded devices
- Additional surveillance modules
- Multi-camera systems

3. SOURCE CODE AND FILE STRUCTURE

3.1 Backend Files

The backend section contains the main Python files responsible for Artificial Intelligence processing, surveillance operations, real-time video handling, and dashboard communication.

app.py

app.py is the main controller file of the project. It manages:

- Flask server initialization
- Camera stream handling
- YOLOv8 object detection
- DeepFace recognition
- Real-time processing workflow
- Dashboard communication
- Multi-threading operations

The file acts as the core processing engine of the surveillance system.

YOLOv8 Detection Module

The YOLOv8 detection module is responsible for:

- Detecting persons
- Detecting aircraft objects
- Generating bounding boxes
- Providing real-time object localization

The model processes video frames continuously for real-time surveillance operations.

DeepFace Recognition Module

The DeepFace module handles:

- Face embedding generation

- Identity comparison
- Face recognition
- Matching detected faces with stored database images

This module enables accurate recognition of registered individuals.

Multi-Threading Implementation

The backend uses Python threading to improve real-time performance. Separate threads are used for:

- Video capture
- AI inference
- Dashboard streaming
- Detection processing

This reduces lag and improves smooth operation during continuous surveillance.

Flask Backend Routes

Flask routes are used to:

- Stream live video
- Handle dashboard requests
- Control surveillance operations
- Process uploaded files
- Manage downloadable log reports

The backend architecture supports modular integration and future scalability.

3.2 Frontend Files

The frontend section contains files responsible for the visual interface and user interaction of the surveillance dashboard.

index.html

The index.html file contains the main structure of the surveillance dashboard. It displays:

- Live video feed
- Detection outputs
- Recognition labels
- Dashboard controls

CSS Styling

CSS files were used to create the dark-themed aerospace-inspired user interface. Styling was applied for:

- Dashboard layout
- Buttons
- Detection panels
- Live monitoring interface

JavaScript Functions

JavaScript was used to implement:

- Start and Stop controls
- Real-time dashboard updates
- File upload handling
- Interactive surveillance operations

Live Video Streaming

The frontend receives processed video streams from Flask and displays them continuously on the dashboard for real-time monitoring.

The frontend design improves usability and provides a centralized monitoring environment.

3.3 Database Structure

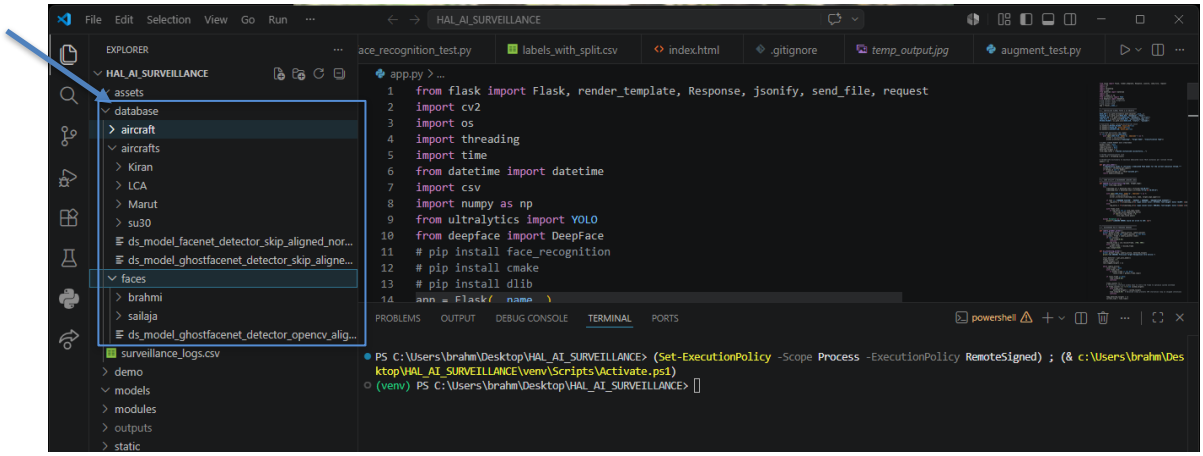
The project uses local database directories to store images used for face recognition and aircraft detection.

database/faces/

This directory stores face images of registered individuals. The images are used by DeepFace for:

- Facial embedding generation
- Identity comparison
- Face recognition operations

Each folder inside the directory represents a specific registered person.



This directory stores military aircraft images such as SU-30, LCA, and KIRAN, etc., used for aircraft detection and recognition experiments.

The database supports:

- Training reference storage
- Detection testing
- Recognition comparisons

Log Data Storage

The system also stores surveillance log data containing:

- Detection names

- Recognition results
- Detection timestamps
- Aircraft detection records

The stored log files can later be downloaded from the dashboard for monitoring and review purposes.

The modular database structure improves:

- Data organization
- Recognition efficiency
- Scalability
- Future system expansion

The complete source code and implementation details are available on GitHub:

https://github.com/Brahmi30/HAL_SURVILIENCE_PROJECT

4. WORKING OF THE SYSTEM

The system workflow was designed to achieve smooth real-time processing, balanced detection accuracy, and efficient surveillance monitoring.

4.1 System Startup Procedure

The surveillance system begins execution by running the main backend application file, app.py.

During startup, the following operations are initialized:

- Flask web server activation
- Camera initialization
- YOLOv8 model loading
- DeepFace model loading
- Multi-threading initialization
- Dashboard route activation

After successful initialization, the user can access the surveillance dashboard through a web browser. The dashboard acts as the main monitoring interface for all surveillance operations.

4.2 Face Recognition Working

The face recognition module is responsible for identifying registered individuals from live video streams.

The working process includes:

- Capturing live video frames
- Detecting persons using YOLOv8
- Extracting face regions from detected persons
- Generating facial embeddings using DeepFace
- Comparing embeddings with stored database images
- Displaying recognized names on the dashboard

If a matching identity is found, the system displays the person's name along with a bounding box around the detected face. If no match is found, the system labels the face as unknown.

The recognition process runs continuously for real-time monitoring applications.

4.3 Aircraft Detection Working

The aircraft detection module uses the YOLOv8 object detection model to identify aircraft objects from the live video stream.

The detection process includes:

- Capturing video frames
- Passing frames to the YOLOv8 model
- Detecting aircraft objects
- Generating bounding boxes around detected aircraft
- Displaying aircraft detection labels in real time

The system continuously monitors incoming frames and updates detections dynamically through the surveillance dashboard.

This module demonstrates the application of Artificial Intelligence in aerospace monitoring and surveillance systems.

4.4 Dashboard Controls and Operations

The surveillance dashboard provides a centralized interface for monitoring and controlling all system operations in real time. The major components of the dashboard are explained below:

Live Surveillance Display Panel:
The live surveillance panel displays the real-time video stream captured from the webcam. Detection results such as bounding boxes, recognition labels, and aircraft detections are displayed directly on the video feed.

Start Surveillance Button:
The Start Surveillance button activates the complete surveillance pipeline by initializing the webcam, AI processing threads, YOLOv8 object detection, and DeepFace recognition modules.

Stop Surveillance Button:
The Stop Surveillance button terminates live monitoring operations, stops video processing, and returns the system to standby mode.

System Status Panel
The system status section displays important monitoring information such as:
• Camera activity status
• Recognition status
• Number of detected targets
• Current surveillance condition

Examples include:
• ACTIVE
• OFFLINE
• SCANNING
• ALERT
• SECURE

File Upload and Recognition Feature (Investigative analysis button)
The dashboard allows users to upload image files manually for face recognition and aircraft detection. Uploaded files are processed through the same AI pipeline used for live surveillance monitoring.

Detection Labels and Bounding Boxes
Detected faces and aircraft objects are highlighted using bounding boxes and corresponding recognition labels for easier monitoring and target identification.

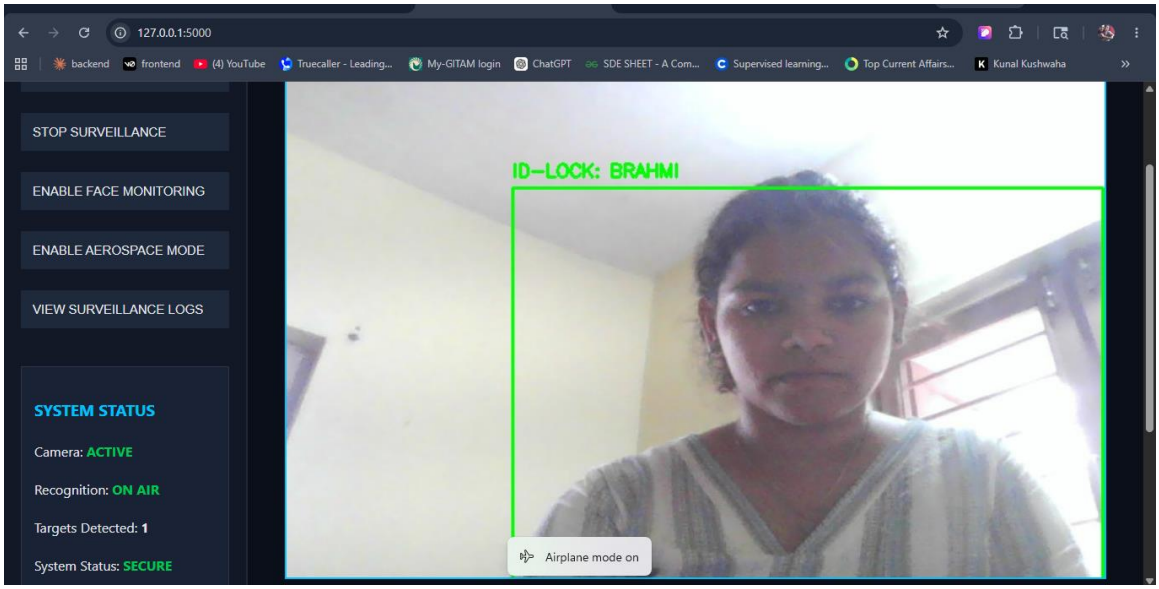
Surveillance Log Display
The dashboard stores surveillance records including:
• Detection names
• Aircraft detections
• Recognition timestamps
• Detection records

These logs are displayed in real time and help maintain surveillance monitoring history.

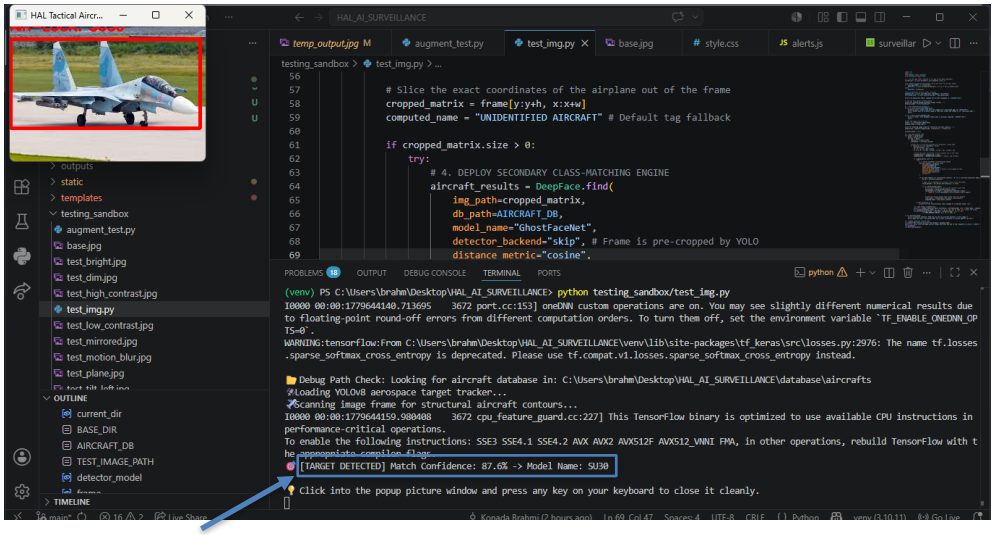
Download Report Feature
The dashboard provides an option to download surveillance logs in CSV format for future analysis, monitoring, and record maintenance.

5. RESULTS

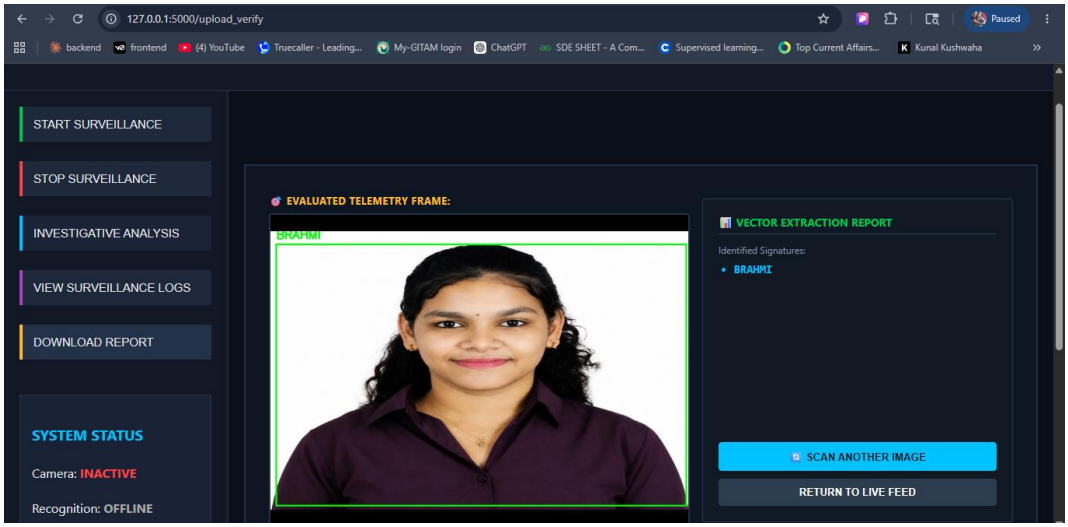
5.1 Face Recognition Output



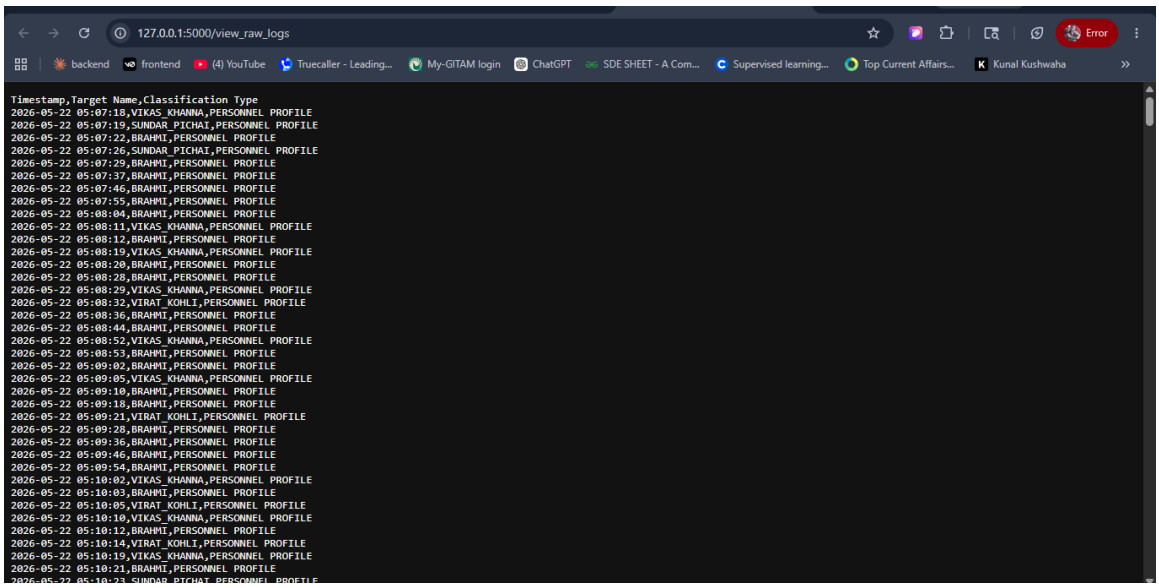
5.2 Aircraft Detection Output



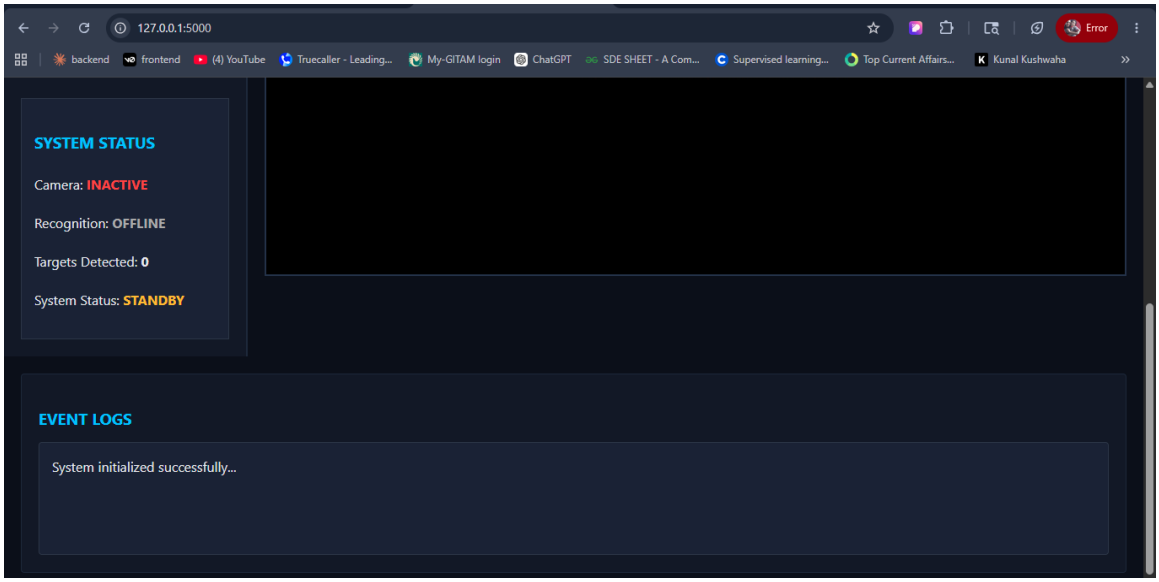
5.3 File Upload and Recognition Output



5.4 Saved Log Details and Reports



5.5 Event logs and System Status



6. Performance Analysis

The system demonstrated smooth real-time video processing with reduced lag through frame resizing, frame skipping, and multi-threading techniques. It successfully recognized registered faces and detected aircraft objects with balanced speed and accuracy. Cropped-region processing and YOLOv8-based optimization helped reduce CPU load and improve overall system efficiency.

7. CHALLENGES FACED DURING DEVELOPMENT

1. Speed vs Accuracy Trade-Off

Balancing detection accuracy and real-time speed was one of the most important engineering challenges during development.

Higher detection accuracy generally required:

- Larger AI models
- More processing time
- Increased computational resources

However, real-time surveillance required:

- Faster processing
- Continuous video streaming
- Low-latency outputs

A balance had to be maintained between: Detection accuracy, Processing speed, System responsiveness

2. Real-Time Processing and CPU Load

Running multiple AI operations such as YOLOv8 detection, DeepFace recognition, dashboard streaming, and video processing simultaneously increased CPU usage during continuous surveillance operations.

To improve system responsiveness, optimization techniques such as frame resizing, frame skipping, cropped-region processing, and multi-threading were implemented to reduce processing overhead and maintain smoother real-time performance.

8. FUTURE SCOPE

- Implementation of the system on embedded devices such as **NVIDIA Jetson** and **Raspberry Pi** for portable AI surveillance applications.
- Integration of advanced AI models for faster and more accurate face recognition and aircraft detection.
- Integration of drone detection and automated alert systems for advanced security and surveillance applications.

9. CONCLUSION

The **Real-Time AI Surveillance System for Face and Aircraft Recognition** was successfully developed using Artificial Intelligence and Computer Vision technologies. The system continuously monitors live video streams and performs real-time face recognition and aircraft detection efficiently.

Several challenges related to real-time processing and optimization were addressed using performance improvements and multi-threading techniques. The final system demonstrates the application of AI in surveillance, aerospace monitoring, and future intelligent security systems.

10. REFERENCES

1. Ultralytics YOLOv8 Documentation – https://docs.ultralytics.com/
2. DeepFace Documentation – https://github.com/serengil/deepface
3. Flask Documentation – https://flask.palletsprojects.com/
4. Python Documentation – https://docs.python.org/3/
5. HAL Official Website – https://hal-india.co.in/